



A TECHNICAL THESIS MANUSCRIPT

AstreaBlue: A Centralized Enterprise IT Service Management, Asset Governance, Endpoint Monitoring, and External Ticket Integration Platform

System Architecture, Role-Based Workflows, Implementation, Validation, and
Development Roadmap

DOCUMENT	DETAIL
Classification	Academic technical thesis and system implementation report
Prepared by	AstreaBlue Project Team
Prepared for	Academic presentation and system defense
Version	1.0 - Print Edition
Date	22 July 2026
System environment	React frontend, Node.js API, PostgreSQL, Railway, Cloudflare R2, native Windows agent

This manuscript documents the implemented repository and automated validation baseline as of 22 July 2026. No production secrets are included.

Document Control and Declaration

This document is intended to serve as a thesis-level technical record of AstreaBlue. It explains the problem addressed by the platform, its users, architecture, workflows, data-governance controls, implementation approach, validation evidence, sprint history, current limitations, and recommended next phases. The manuscript may be printed directly on A4 paper.

REVISION	DATE	DESCRIPTION	STATUS
1.0	22 July 2026	Initial consolidated technical thesis based on the implemented repository	Approved for presentation

Declaration of technical accuracy

The document separates implemented capability from pilot-stage and planned capability. Statements describing system behavior were derived from the current source tree, database migrations, internal technical documentation, and the automated test baseline. No production secrets, employee passwords, API keys, encryption keys, or database credentials are reproduced.

ROLE	NAME / SIGNATURE	DATE
Researcher / Developer	_____	_____
Technical Adviser	_____	_____
Project Mentor	_____	_____

Abstract

AstreaBlue is a centralized enterprise information technology service management platform designed to connect incident handling, service requests, asset governance, configuration management, employee lifecycle processes, endpoint monitoring, privacy consent, reporting, and external-system ticket intake. The project addresses fragmented support workflows in which incidents, hardware assignments, employee transitions, and monitoring evidence are recorded in separate tools or manual documents. Its principal design decision is to maintain one PostgreSQL-backed operational source of truth while enforcing role, branch, ownership, and machine-identity boundaries at the backend.

The platform uses a React and Vite single-page frontend, an Express and Node.js API, JSON Web Token authentication for human users, per-system API keys for external integrations, and unique per-device credentials for Windows endpoints. A native .NET Windows service provides heartbeat, hardware inventory, software inventory, policy synchronization, screenshot capture, and USB/DLP metadata collection. Privacy-sensitive monitoring is controlled by employee consent and an effective policy that is revalidated by the backend. Screenshots are encrypted with AES-256-GCM before private Cloudflare R2 storage. External systems submit tickets through a centralized gateway without receiving access to the AstreaBlue user interface or internal database.

The implementation follows an iterative, risk-controlled methodology. Schema migrations, modular services, RBAC filters, idempotent integration contracts, audit histories, and automated tests are used to reduce regression risk. At the documented baseline, the backend test suite reports 77 passing tests. Native enrollment, heartbeat, inventory, screenshot capture, and USB/DLP behavior have first-pilot evidence; second-device restart testing, trusted code signing, a signed MSI, and complete retirement of the legacy global monitoring token remain planned. The resulting platform demonstrates how service management, asset custody, privacy governance, and endpoint telemetry can be integrated without erasing the organizational boundaries between SuperAdmin, Admin, Technician, Employee, HR, and external systems.

Keywords: enterprise ITSM, centralized ticketing, asset management, endpoint monitoring, RBAC, consent management, RA 10173, CMDB, DLP, employee lifecycle, external API.

Table of Contents

Document Control and Declaration	2
Declaration of technical accuracy	2
Abstract	3
Table of Contents	4
List of Figures and Tables	7
Chapter 1 - Introduction	8
1.1 Background of the Study	8
1.2 Statement of the Problem	8
1.3 General Objective	8
1.4 Specific Objectives	9
1.5 Scope and Delimitations	9
1.6 Significance of the Study	9
Chapter 2 - Conceptual and Regulatory Foundations	10
2.1 IT Service Management as an Integrated Control System	10
2.2 Privacy and Consent by Design	10
2.3 Least Privilege and Backend Authorization	10
2.4 Auditability and Accountability	10
2.5 Machine Identity	10
Chapter 3 - Development Methodology and System Architecture	11
3.1 Development Method	11
3.2 Technology Stack	11
3.3 Logical Architecture	12
3.4 Backend Route Organization	12
3.5 Data Architecture	12
3.6 Realtime Update Strategy	13
Chapter 4 - Users, Roles, and Access-Control Model	14
4.1 Stakeholder and User-Role Matrix	14
4.2 RBAC Enforcement Principles	14
4.3 Recommended Invitation Governance	15

Chapter 5 - Functional Modules and End-to-End Workflows	16
5.1 Functional Module Inventory	16
5.2 Internal Service Desk Workflow	17
Incident Management	17
Service Request Management	18
Knowledge Base	18
Ticket Schedule Calendar	18
SLA Management	18
5.3 External Ticket Gateway	19
5.4 Asset Management Workflow	20
Hardware Asset Tracking	20
Software License Management	21
Asset Discovery and Reconciliation	21
Depreciation and Finance	21
5.5 Configuration Management and Change Impact	21
5.6 Consent-Aware Endpoint Management	22
Heartbeat, Inventory, and Activity	22
Screenshot Monitoring	23
USB and DLP	23
Health Interpretation	23
5.7 Employee Lifecycle Management	23
5.8 Reporting, Forecasting, and Export	26
Chapter 6 - Security, Privacy, and Reliability Controls	27
6.1 Authentication	27
6.2 Authorization	27
6.3 Privacy Controls	27
6.4 Data Integrity and Transaction Boundaries	27
6.5 Availability and Failure Handling	28
6.6 Secret Management	28
Chapter 7 - Testing, Results, and Evaluation	29
7.1 Validation Strategy	29
7.2 Automated Test Baseline	29
7.3 Physical Pilot Evidence	29
7.4 Acceptance Criteria	29
7.5 Observed Limitations	30

Chapter 8 - Sprint History and Phase Status	31
8.1 Development Phases	31
8.2 Native Agent Delivery Priorities	31
Chapter 9 - Conclusions and Recommended Roadmap	32
9.1 Conclusions	32
9.2 Priority Roadmap	32
9.3 Final Recommendation	32
References	33
Appendix A - Demonstration Script	34
Appendix B - External Ticket API Example	35
Appendix C - Operational Acceptance Checklist	36
Appendix D - Acronyms and Terms	37

List of Figures and Tables

TYPE	ITEM
Figure 1	AstreaBlue logical deployment architecture
Figure 2	Internal Service Desk ticket workflow
Figure 3	External Ticket Gateway workflow
Figure 4	Asset and replacement workflow
Figure 5	Consent-aware endpoint monitoring workflow
Figure 6	Employee onboarding workflow
Figure 7	Employee offboarding workflow
Table 1	Stakeholder and user-role matrix
Table 2	Functional module inventory
Table 3	SLA target matrix
Table 4	Sprint and phase status
Table 5	Validation baseline and open acceptance risks

Chapter 1 - Introduction

1.1 Background of the Study

Organizations with multiple internal systems commonly experience a fragmented support environment. Employees report problems through informal messages, branch technicians see incomplete context, assets are tracked separately from incidents, and externally developed systems cannot create standardized support records. Endpoint telemetry may exist but remain disconnected from employee consent, hardware assignment, and incident response. These conditions make it difficult to establish accountability, measure service performance, and prove who changed an operational record.

AstreaBlue was conceived as a unified IT service management platform. It consolidates support demand, technology assets, configuration dependencies, endpoint evidence, and employee lifecycle controls while retaining strict access boundaries. The platform is not merely a ticket form: it is a coordinated set of workflows in which events create traceable records and authorized actions update related operational data.

1.2 Statement of the Problem

The project addresses the following core problems:

- Support requests from employees and independent company systems do not naturally enter one controlled queue.
- Branch-based operations require visibility boundaries that cannot safely depend on frontend filtering alone.
- Hardware, software licenses, endpoint telemetry, and employee responsibility are often recorded as unrelated data.
- Privacy-sensitive monitoring requires documented consent, policy enforcement, retention, and auditable administrative control.
- Onboarding and offboarding require both human verification and automatic evidence without allowing checklists to falsely represent completion.
- Management reporting requires consistent data, SLA calculations, and exports derived from the same source of truth.

1.3 General Objective

To design, implement, and validate a centralized enterprise ITSM platform that integrates Service Desk operations, asset and configuration governance, endpoint monitoring, employee lifecycle workflows, reporting, and external ticket intake under enforceable role-based and branch-based access control.

1.4 Specific Objectives

- Provide standardized incident and service-request lifecycles with assignment, comments, attachments, notifications, history, and SLA evidence.
- Enable approved external systems to create and follow tickets through a secure, idempotent backend-to-backend API.
- Maintain hardware, software-license, discovery, financial, and configuration-item records with operational relationships.
- Operate a native Windows endpoint agent using unique device credentials, consent-derived policy, secure local credential storage, and offline-tolerant telemetry delivery.
- Support repair and replacement workflows that update asset custody while preserving device identity and historical evidence.
- Provide pre-hire onboarding and controlled offboarding workflows with role separation, verification gates, and internal automation.
- Present management analytics and consistent exports without exposing data outside authorized scope.

1.5 Scope and Delimitations

The documented system covers the AstreaBlue web application, backend API, PostgreSQL schema, Cloudflare R2 screenshot storage, SMTP-dependent notifications, and native Windows agent. External HRIS, payroll, accounting, e-invoicing, and inventory applications remain independent systems; AstreaBlue provides their ticket contract and credentials but does not require access to their source code. Offboarding automations are intentionally limited to records controlled by AstreaBlue and do not deactivate external email, VPN, HRIS, or cloud accounts.

Important boundary

AstreaBlue records and verifies IT actions. It does not claim to control third-party systems unless a future, separately authorized connector is implemented.

1.6 Significance of the Study

Employees gain a single, traceable support channel. Technicians receive branch-appropriate work queues and technical context. Administrators gain approval, asset, consent, and lifecycle controls. SuperAdministrators gain company-wide governance, integrations, and audit visibility. HR gains a restricted lifecycle workspace without broad IT administration. Management gains consistent analytics. External development teams gain a minimal contract: a production API URL, a unique system credential, and a documented request schema.

Chapter 2 - Conceptual and Regulatory Foundations

2.1 IT Service Management as an Integrated Control System

The project treats ITSM as a set of connected records rather than independent screens. A ticket can be linked to an employee, branch, category, priority, SLA, asset, configuration item, replacement request, endpoint alert, or external system. This relationship-centered model allows operational evidence to be reused across dashboards and audits.

2.2 Privacy and Consent by Design

Republic Act No. 10173 establishes the Philippine Data Privacy Act of 2012 and frames the protection of personal information in information and communications systems [1]. AstreaBlue applies this concern through visible consent documents, explicit monitoring categories, approval state, effective device policy, administrative audit history, retention controls, and the ability for an authorized SuperAdmin to pause screenshot collection. This implementation is a technical control framework and does not replace organizational legal review.

2.3 Least Privilege and Backend Authorization

The security model uses deny-by-default reasoning: possession of a valid login does not automatically authorize access to every record. Human access is checked by role, branch, ownership, assignment, ticket origin, and sensitivity. This aligns with OWASP guidance that object-level and function-level authorization must be consistently enforced in APIs, particularly for administrative functions [2].

2.4 Auditability and Accountability

Operational changes are represented as history events rather than silent overwrites wherever practical. Tickets, lifecycle cases, replacement requests, consent actions, asset assignments, endpoint policies, and integration calls retain actor and timestamp evidence. NIST SP 800-53 identifies audit and accountability as a dedicated control family, supporting the decision to preserve histories and restrict destructive deletion [3].

2.5 Machine Identity

AstreaBlue separates human identity from machine identity. Human users authenticate with a JWT-backed session. External systems use individually provisioned API keys stored as hashes. Windows devices enroll with short-lived single-use codes and receive unique credentials bound to their permanent device UUID. Windows DPAPI protects device credentials at rest; Microsoft documents DPAPI as an operating-system service for encrypting data using user or machine credentials [4].

Chapter 3 - Development Methodology and System Architecture

3.1 Development Method

The project follows iterative delivery with risk-based verification. Each sprint establishes a bounded capability, adds database migration support, preserves existing routes and role rules, and is followed by syntax checks, focused tests, full backend regression tests, and a frontend production build. High-risk areas such as ticket visibility, consent, device credentials, assignment, replacement, lifecycle closure, and external API idempotency receive dedicated tests.

3.2 Technology Stack

LAYER	TECHNOLOGY	RESPONSIBILITY
Frontend	React 18, Vite, React Router, Zustand, Recharts, Socket.IO client	Role-aware UI, dashboards, forms, tables, live invalidation
Backend	Node.js, Express 5, JWT, bcrypt, Multer, Nodemailer, Socket.IO	REST API, authentication, domain workflows, notifications, realtime events
Database	PostgreSQL	Transactional source of truth, RBAC-scoped queries, audit records, migrations
Object storage	Cloudflare R2 through S3-compatible SDK	Private encrypted screenshot objects and retention deletion
Endpoint	.NET Framework native Windows service and companion	Enrollment, heartbeat, inventory, policy, activity, screenshot, USB/DLP metadata
Deployment	Railway-hosted frontend and backend	Production web delivery and environment-managed secrets

3.3 Logical Architecture

AstreaBlue logical deployment architecture

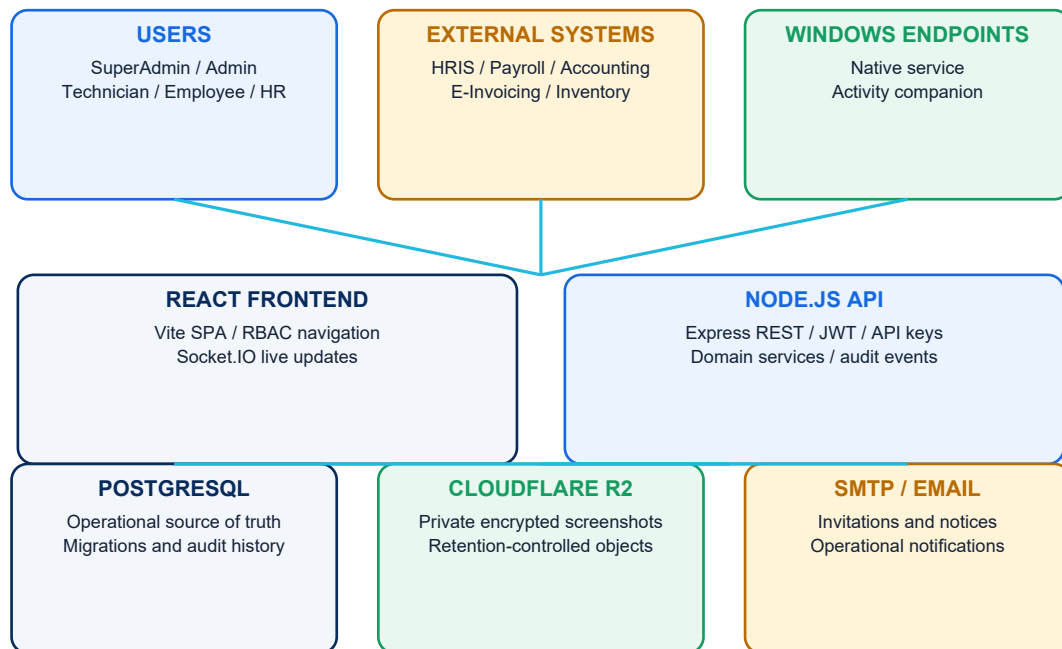


Figure 1. AstreaBlue logical deployment architecture.

The React frontend never connects directly to PostgreSQL or R2. It calls authenticated backend endpoints. External systems call only the external ticket gateway from trusted server-side code. Windows devices use their own credential path. The backend performs policy and authorization checks before accessing operational data or encrypted objects.

3.4 Backend Route Organization

The API is progressively modularized under backend/src/routes. Active domains include authentication, tickets, attachments, SLA, assets, consent, endpoint management, integrations, employee lifecycle, replacement requests, CMDB, analytics, projects, calendar, reports, invitations, users, roles, branches, notifications, and knowledge base. The legacy server.js remains a mixed bootstrap and compatibility layer; the refactor roadmap therefore prioritizes gradual extraction rather than high-risk wholesale deletion.

3.5 Data Architecture

PostgreSQL is the authoritative transactional store. Domain tables represent users and roles, branches, tickets and comments, SLA policies and histories, hardware assets, software licenses, monitored devices, endpoint inventory, consent documents, effective policies, screenshot metadata, USB/DLP events, configuration items and relationships, replacement requests, lifecycle cases, external integrations, and audit logs. Database migrations are idempotent and are executed before server startup.

3.6 Realtime Update Strategy

Socket.IO events provide low-latency invalidation for tickets and selected operational modules. The durable database remains authoritative: reconnecting clients re-fetch the current server state. This hybrid model avoids making WebSocket delivery the sole source of truth and prevents missed events from permanently corrupting a dashboard.

Chapter 4 - Users, Roles, and Access-Control Model

4.1 Stakeholder and User-Role Matrix

ROLE	PRIMARY RESPONSIBILITY	DATA SCOPE	KEY RESTRICTIONS
SuperAdmin	Company-wide governance, system configuration, integrations, users, audit, global operations	All branches and centralized external tickets	Sensitive actions remain audited; completed lifecycle records are protected
Admin	Branch administration, ticket oversight, consent approval, assets, lifecycle IT actions	Assigned branch	No global integration secrets or cross-branch access
Technician	Accept and resolve eligible incidents, contribute technical KB content, endpoint operations as permitted	Assigned branch and assigned/available eligible tickets	Cannot see centralized external tickets or privacy/role-change/lifecycle-sensitive tickets
Employee	Submit and follow own tickets, sign consent, view own records, request repair/replacement	Own identity, tickets, consent, and assigned assets	Cannot administer other users, branches, devices, or tickets
HR	Create and oversee branch-scoped onboarding/offboarding cases and HR-owned verification	Assigned branch lifecycle cases and permitted linked tickets	Cannot complete IT-owned tasks or invite privileged roles
External System	Submit and follow tickets through machine API	Only its own external references and tickets	No web login, no internal IDs, no cross-system access
Windows Device	Report telemetry and retrieve effective policy	One enrolled device identity	Credential bound to device; policy and consent revalidated server-side

4.2 RBAC Enforcement Principles

- Backend query scoping is authoritative; hidden buttons are only a usability layer.
- SuperAdmin is the only human role with company-wide external-ticket visibility.
- Admin and Technician access is branch-scoped, with additional category and assignment restrictions.
- Employee access is ownership-scoped.
- HR access is lifecycle-focused and does not imply broad IT administration.
- Sensitive consent, privacy, role-change, and lifecycle-linked tickets are excluded from ordinary technician queues.
- Invitation endpoints must hard-code or validate the maximum assignable role to prevent privilege escalation.

4.3 Recommended Invitation Governance

The onboarding invitation is intended only for the Employee role and should be tied to a lifecycle case and its branch. HR may be permitted to initiate or resend that employee-only invitation if the backend fixes the role to Employee and the branch to the case. Invitations for Technician, Admin, HR, or other privileged roles should remain in SuperAdmin User and Role Management. This separation keeps routine onboarding efficient without allowing HR to grant administrative privileges.

Chapter 5 - Functional Modules and End-to-End Workflows

5.1 Functional Module Inventory

MODULE	SUBMODULES / FUNCTIONS	PRIMARY OUTPUT
Dashboard and Analytics	Executive dashboard, operational analytics, asset/endpoint analytics, governance, forecasting, custom reports	KPIs, trends, drill-down reports
Service Desk	Incident Management, Service Request Management, Knowledge Base, Calendar, SLA	Traceable ticket lifecycle
Asset Management	Hardware, software licenses, discovery/reconciliation, depreciation and finance	Custody, inventory, lifecycle and financial evidence
Configuration Management	Configuration items, dependency map, change impact analysis	Service dependency and blast-radius evidence
Replacement Management	Employee request, assessment, repair path, replacement path, asset exchange	Controlled repair/replacement and history
Endpoint Management	Devices, inventory, health, activity, screenshots, USB/DLP, policy, administration	Consent-aware endpoint evidence
Employee Lifecycle	Pre-hire onboarding, account invitation, evidence checklist, offboarding automation	Verified joiner/leaver records
Integration Hub	Registered systems, API keys, console, API logs	External-system ticket connectivity and audit
System Administration	Users, roles, branches, configuration, email diagnostics	Controlled platform administration

5.2 Internal Service Desk Workflow

Internal Service Desk ticket workflow

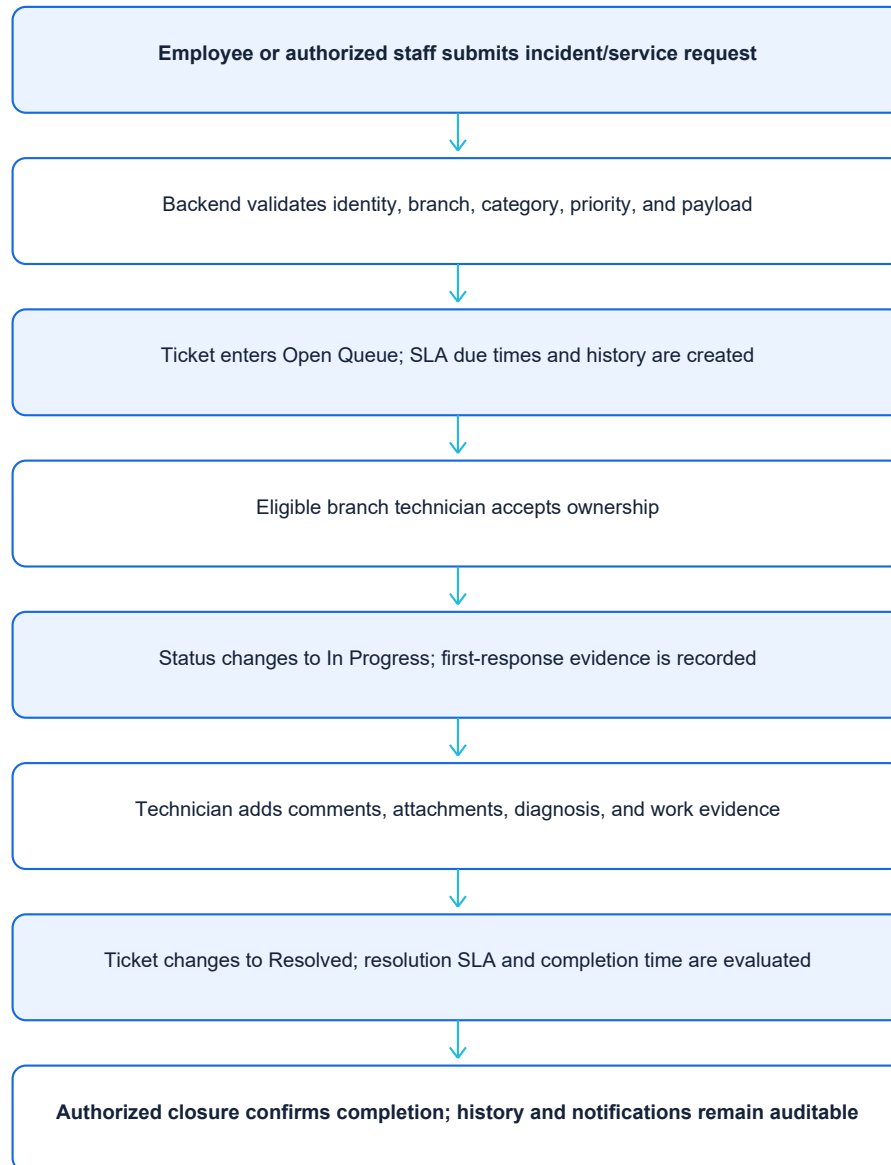


Figure 2. Internal Service Desk ticket workflow.

Incident Management

Incident Management handles unplanned interruptions or degraded service. Tickets carry canonical numbers, priority P1 through P4, status, branch, category, requester, assignment, comments, attachments, SLA deadlines, work-started time, resolved time, completion duration, and audit history. Admin and SuperAdmin may correct priority when a requester misjudges urgency, while the change remains attributable.

Service Request Management

Service requests represent planned assistance such as access, equipment, information, or standard fulfillment. They share the core ticket engine but can be categorized and routed differently from incidents. The design avoids a second disconnected ticket database.

Knowledge Base

The Knowledge Base captures reusable solutions and operational guidance. Technician article creation should offer only tickets the technician accepted or resolved, preventing unrelated branch tickets from being used as article sources. Articles support faster resolution and consistent troubleshooting.

Ticket Schedule Calendar

The calendar presents ticket events and due dates in a time-oriented view. Role and branch filters must match the Service Desk access model. Calendar failures must never broaden access; a server error is preferable to returning unauthorized records.

SLA Management

SLA timing is based on the active priority policy at ticket creation. First response is measured from creation until work first enters In Progress. Resolution is measured from creation until Resolved or Closed. Cancellation changes both targets to Cancelled rather than Met or Breached. The current seeded targets are shown below.

PRIORITY	FIRST RESPONSE	RESOLUTION	INTERPRETATION
P1 - Critical	15 minutes	120 minutes	Major operational interruption requiring immediate attention
P2 - High	30 minutes	240 minutes	High business impact with urgent restoration
P3 - Medium	120 minutes	480 minutes	Normal operational issue
P4 - Low	240 minutes	1,440 minutes	Low urgency or limited impact

5.3 External Ticket Gateway

External Ticket Gateway workflow

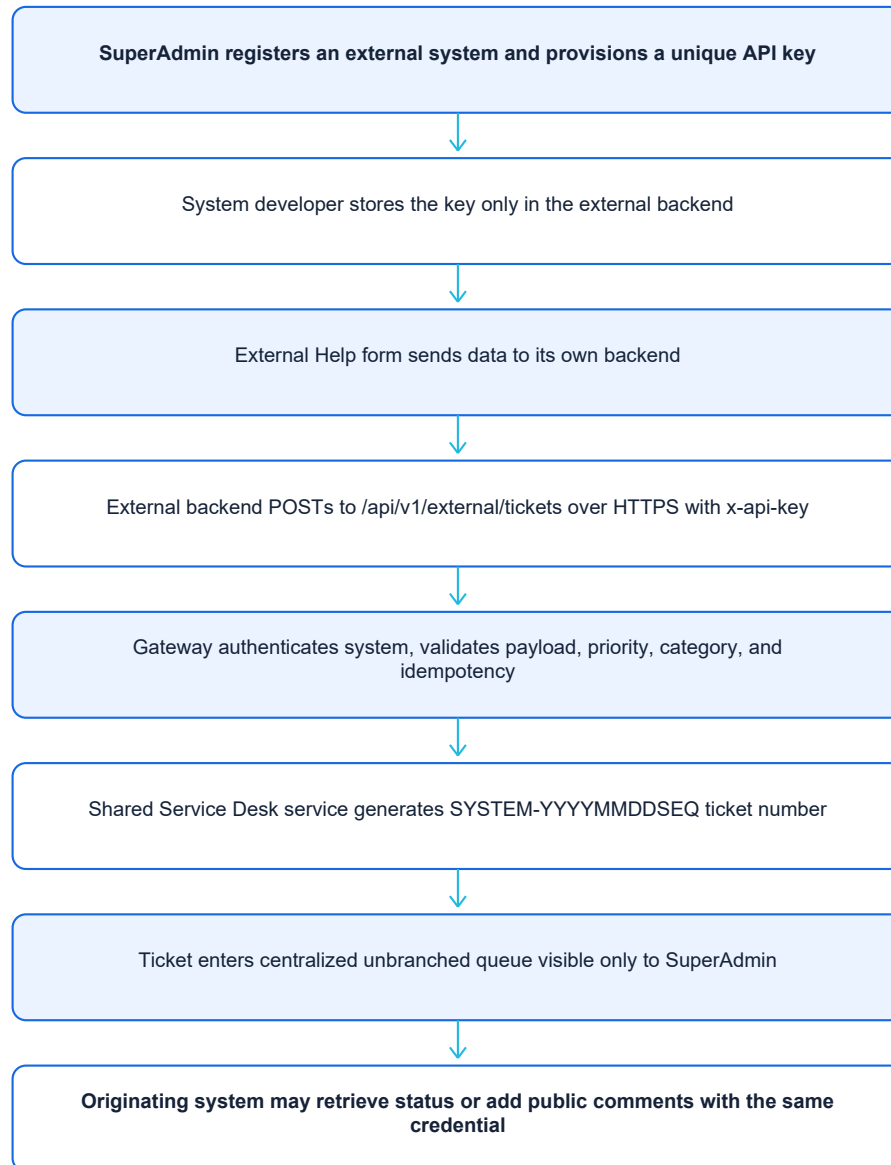


Figure 3. External Ticket Gateway workflow.

The gateway requires a stable external_reference. Replaying the identical logical request returns the existing ticket, while a conflicting reuse returns HTTP 409. Keys are displayed only at issuance and stored as SHA-256 hashes. Each integration is isolated from every other integration. The gateway is backend-to-backend; placing a key in browser code would invalidate the security model.

Current production endpoint:
<https://backend-production-fc059.up.railway.app/api/v1/external/tickets>. Required data includes external employee identifier, requester name and email, origin system and module, external reference, title, and description. Optional category and priority must use known canonical values.

5.4 Asset Management Workflow

Asset and replacement workflow

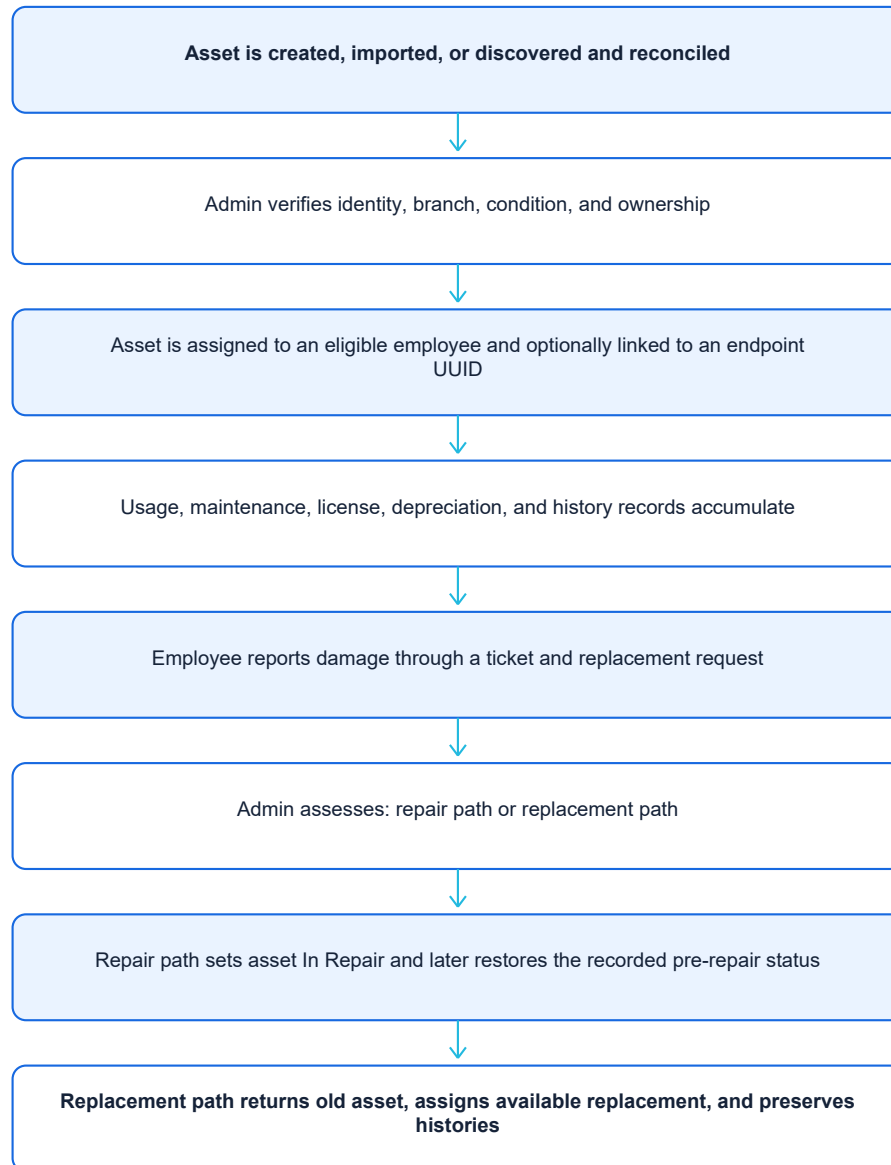


Figure 4. Asset and replacement workflow.

Hardware Asset Tracking

Hardware records include asset tag, type, make/model, serial number, branch, assignment, status, condition, images, financial information, and history. Asset status is an operational state, not merely a label: replacement and offboarding workflows update it transactionally.

Software License Management

License records track product, key or entitlement metadata, branch, seat counts, assignments, purchase and expiry dates, and renewal history. Expiry status is derived from dates. Renewal should create a history record, require a later expiry date, update cost when applicable, and return released seats to the available pool during offboarding.

Asset Discovery and Reconciliation

Discovery records identify observed devices and compare them with managed asset records. Matched means the discovery identity already resolves to an asset; selection controls should therefore represent review or relinking, not imply that an unmatched action remains required. Hardware and software inventory from the native agent are separate evidence streams linked by monitored device identity.

Depreciation and Finance

Financial reporting uses acquisition cost, capitalization data, useful life, depreciation method, accumulated depreciation, and net book value. Reports should state branch/company scope and generation timestamp, and exports should use non-editable or protected formats when distributed as official evidence.

5.5 Configuration Management and Change Impact

Configuration Management records technology components and the relationships among them. CI type describes the technical class, such as Application, Database, Server, Network Device, or Service. Category supplies a business or operational grouping; it should not duplicate CI type. Supported relationships include depends on, connected to, uses, hosts, runs on, contains, and linked to.

Change Impact Analysis traverses CI relationships. Upstream dependencies are components the selected CI needs; downstream or affected CIs are components that depend on the selected CI. Related branches come from the selected and connected records. Dependent application counts and the impact score are calculated from graph reach, relationship direction, CI criticality, environment, and branch spread. Low, Medium, or High risk is therefore a derived classification, not a manually typed label.

EXAMPLE CI	RELATIONSHIP	TARGET CI	WHY IT MATTERS
Inventory Web Application	depends on	Inventory PostgreSQL Database	Database outage directly affects the application
Inventory Web Application	uses	AstreaBlue External Ticket API	Help submissions require gateway availability
Application Server	hosts	Inventory Web Application	Server maintenance affects hosted service

5.6 Consent-Aware Endpoint Management

Consent-aware endpoint monitoring workflow

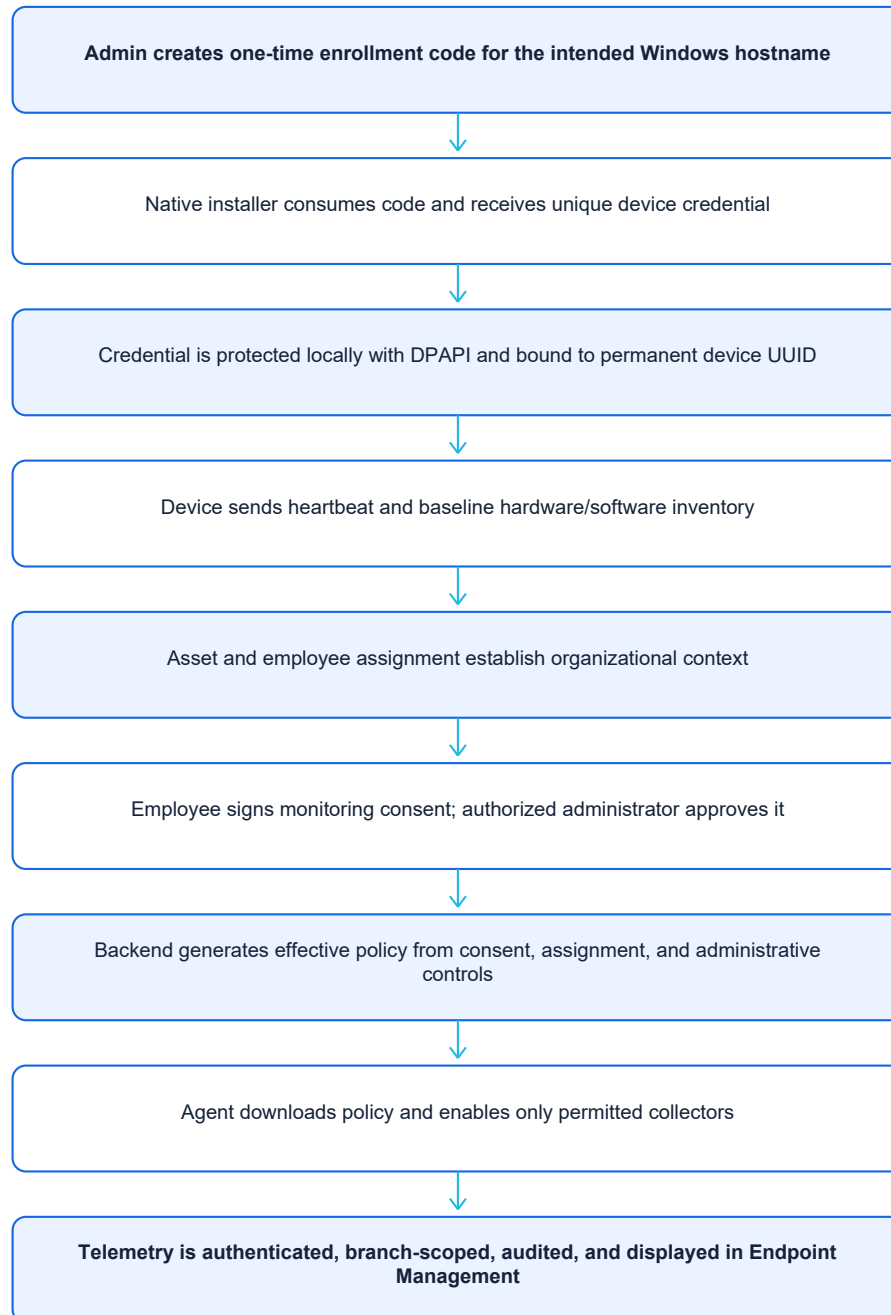


Figure 5. Consent-aware endpoint monitoring workflow.

Heartbeat, Inventory, and Activity

The Windows service starts automatically and sends heartbeats at the configured interval. Hardware and software inventory run periodically, presently on a 24-hour schedule when enabled. The interactive companion observes foreground application, active window title, and idle duration; the service forwards these samples only when the effective policy permits activity monitoring.

Screenshot Monitoring

The interactive companion displays a Windows notification before capture. The service uploads through its authenticated channel. The backend revalidates device assignment, consent, and policy; encrypts the image with AES-256-GCM; stores ciphertext in private R2; retains metadata in PostgreSQL; restricts viewing by RBAC and branch; and deletes expired objects according to retention policy. A SuperAdmin pause/resume control updates effective policy without revoking device identity.

USB and DLP

The agent records removable-drive insertion/removal and metadata for files written or changed on the removable volume. File contents are not collected. Server-side rules assign risk from attributes such as file extension, sensitivity pattern, size, count, transfer volume, and policy thresholds. High or Critical events can create alerts and, when policy permits, automatic Service Desk incidents. An offline queue resubmits events idempotently after connectivity returns.

Health Interpretation

Online status indicates recent heartbeat, not full monitoring health. A device can be Online while inventory is stale or privacy-sensitive collectors are disabled. Warning states therefore identify evidence that has exceeded its freshness threshold. Repairing or updating the agent should preserve the device UUID unless the device is intentionally purged and re-enrolled as a new machine.

5.7 Employee Lifecycle Management

Employee Lifecycle separates HR oversight from IT implementation without creating a second identity system. A pre-hire onboarding case can exist before the employee has an AstreaBlue account. Checklists contain both manually verified organizational tasks and automatically reconciled evidence tasks. The case cannot be completed while required evidence remains pending.

Employee onboarding workflow

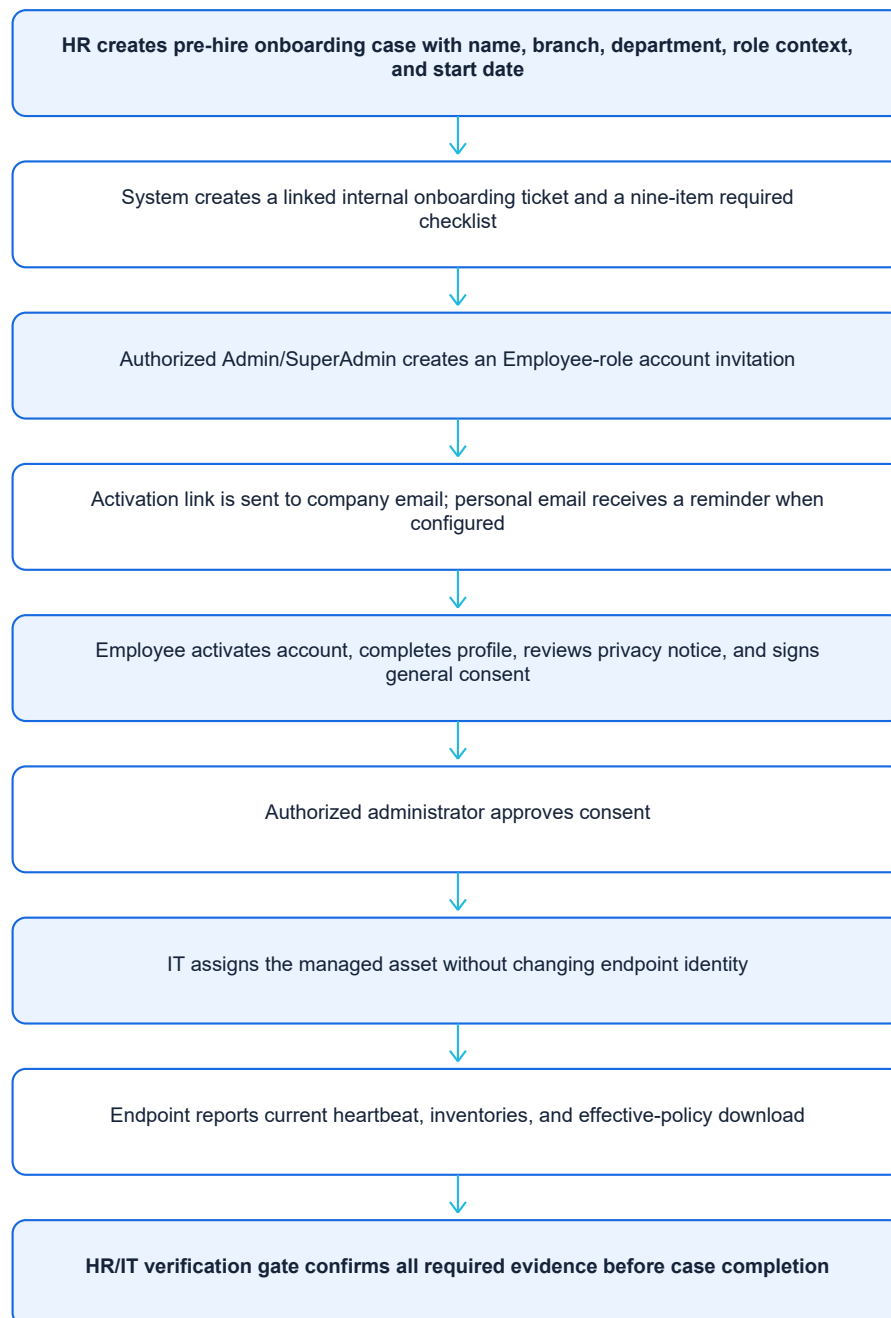


Figure 6. Employee onboarding workflow.

Employee offboarding workflow

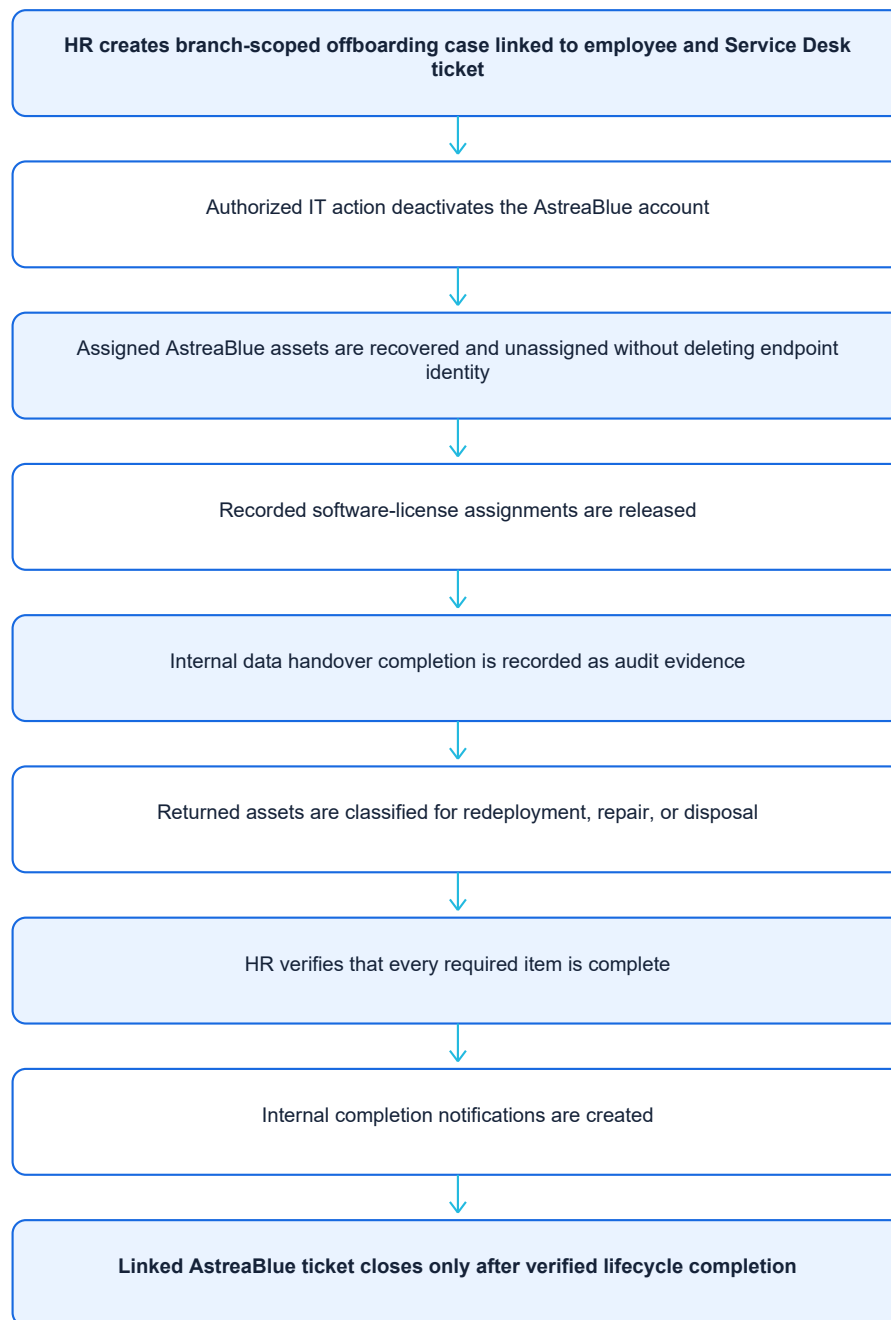


Figure 7. Employee offboarding workflow.

Evidence rule

Automatically verifiable onboarding items cannot be falsely checked by hand. Account, consent, assignment, heartbeat, inventory, and policy evidence must exist in the system.

5.8 Reporting, Forecasting, and Export

Executive and operational dashboards aggregate Service Desk, SLA, asset, endpoint, compliance, and project data from the same PostgreSQL source. Filters must be applied before aggregation and export. Official exports should include the AstreaBlue logo, report title, branch or company scope, generation timestamp, table/list format, and page numbering. Supported standardized outputs are Excel, TXT, and PDF; PDF is the non-editable presentation record, while Excel may be protected to discourage accidental modification.

Chapter 6 - Security, Privacy, and Reliability Controls

6.1 Authentication

ACTOR	AUTHENTICATION	SECRET STORAGE	REVOCATION
Human user	Email/password and JWT	Password hash in PostgreSQL; JWT held by client	Deactivate account / expire session
External system	x-api-key	Only SHA-256 hash stored by AstreaBlue	Disable system/key or rotate credential
Windows endpoint	Unique device credential	Hash in PostgreSQL; DPAPI-protected value on laptop	Revoke/rotate device credential

6.2 Authorization

Authorization is applied at route and query level. Record identifiers supplied by clients are treated as untrusted. Ticket access uses canonical user context rather than accepting a claimed role from a query string. Integration calls are scoped to the authenticated system. Endpoint requests are scoped to the credential-bound device. This directly addresses broken object- and function-level authorization risks identified by OWASP [2].

6.3 Privacy Controls

- General employee consent supplies one approved preference record for assigned managed devices.
- Effective policy intersects consent, assignment, administrative controls, and safe defaults.
- Screenshot and USB/DLP collection remain disabled without explicit relevant consent.
- Screenshot notification, encryption, private storage, access audit, pause/resume, and retention are enforced.
- USB file contents are not uploaded; the system records operational metadata only.
- Consent changes, approvals, withdrawals, and supersession remain auditable.

6.4 Data Integrity and Transaction Boundaries

Multi-record workflows use PostgreSQL transactions and row locks. Replacement exchange updates the old asset, replacement asset, monitored-device assignment, assignment history, and request history as one controlled operation. Lifecycle closure verifies pending tasks before changing final state. External ticket idempotency prevents network retries from creating duplicate incidents.

6.5 Availability and Failure Handling

The native agent uses service recovery, rotating local logs, and offline queues for USB/DLP delivery. WebSocket loss does not erase data because clients re-fetch durable REST state. Database readiness gates avoid serving feature routes before migrations complete. SMTP failure reports delivery failure without pretending that an email was sent; invitation links can still be securely copied while configuration is corrected.

6.6 Secret Management

Railway environment variables hold production database, SMTP, JWT, integration, R2, and screenshot encryption configuration. Secrets must not be committed, embedded in frontend bundles, printed in logs, or included in screenshots and support messages. Exposed keys must be rotated rather than merely hidden.

Chapter 7 - Testing, Results, and Evaluation

7.1 Validation Strategy

Validation combines static syntax checks, domain unit tests, route-level integration tests, frontend production builds, migration execution, and physical-device pilot checks. The complete backend suite is executed serially to reduce shared-database interference. High-risk authorization cases are explicitly tested rather than inferred from successful UI behavior.

7.2 Automated Test Baseline

TEST DOMAIN	VALIDATED BEHAVIOR
Ticket RBAC	Employee ownership, branch technician scope, HR scope, SuperAdmin external-ticket exclusivity
External Gateway	Authentication, validation, canonical priority, idempotency, cross-system isolation
Endpoint Enrollment	Single-use codes, unique credentials, revocation, DPAPI-at-rest smoke behavior
Native Agent	Policy, heartbeat, hardware/software inventory, screenshot and USB/DLP smoke paths
Consent and Onboarding	Approval reconciliation, mandatory evidence, role restrictions, lifecycle completion gates
Replacement	Repair lifecycle, pre-repair status restoration, assignment preservation
Screenshot Retention	R2 object deletion, metadata removal, audit record
Reports	Branded Excel, TXT, and valid PDF output
Realtime	Ticket Socket.IO events and durable refresh behavior

Automated result

Backend regression baseline: 77 tests passed, 0 failed, 0 skipped. Frontend production build completed successfully at the documented repository state.

7.3 Physical Pilot Evidence

CAPABILITY	FIRST PILOT	SECOND / RESTART ACCEPTANCE
Enrollment, heartbeat, diagnostics, repair	Validated	Second-laptop and restart coverage still required
Hardware/software inventory	Validated	Restart and long-running freshness validation required
Activity companion	Validated after repair corrected local access	Repeat across user logon/restart
Encrypted screenshot	Validated on first pilot	Validate repair/restart on second pilot
USB insertion/removal and file metadata	Validated on first pilot	Validate DLP/auto-incident on testing laptop

7.4 Acceptance Criteria

- No user can retrieve data outside the backend-enforced role/branch/ownership scope.
- External systems can create, retrieve, and comment only on their own tickets.

- A device cannot authenticate with a revoked credential.
- Privacy-sensitive telemetry is rejected when consent or effective policy is absent.
- Lifecycle and replacement transitions cannot skip required gates.
- Reports reflect active filters and identify scope and generation time.
- Automated tests and production builds pass before deployment.

7.5 Observed Limitations

The current production architecture remains a modularizing monolith; `server.js` still contains legacy compatibility logic. Native installers and executables are not yet signed with a trusted company certificate. The MSI package, production signed-update manifest, staged update channels, complete multi-laptop restart validation, and removal of the legacy global monitoring token remain incomplete. SMTP delivery depends on valid provider configuration. Browser and network conditions can interrupt realtime connections, requiring REST reconciliation.

Chapter 8 - Sprint History and Phase Status

8.1 Development Phases

PHASE / SPRINT	OBJECTIVE	STATUS	KEY RESULT
Phase 0 - Discovery and safety baseline	Inventory code, routes, RBAC, schema, and critical regressions	Completed	Risk areas identified; non-destructive migration approach adopted
Phase 1 - Core validation	Authentication, RBAC, assets, endpoints, consent, policy, diagnostics	Completed with ongoing hardening	Core human workflows and branch scope operational
Phase 2 - External Ticket Gateway	Centralized API for independent company systems	Implemented	Per-system keys, canonical tickets, idempotency, logs, documentation
Phase 3 - External onboarding	Connect HRIS, payroll, accounting, e-invoicing, inventory Help pages	Foundation ready; partner adoption in progress	External teams require only API URL, key, and contract
Phase 4 - Endpoint monitoring	Activity, screenshots, USB/DLP, automatic incidents	Pilot implemented	Consent-aware native collectors, encrypted R2, DLP scoring
Sprint 5 - CMDB and analytics	CI relationships, impact, dashboards, exports	Implemented; UX/performance tuning ongoing	Dependency-aware reporting and impact views
Sprint 6 - Replacement Management	Repair and laptop exchange with asset updates	Implemented	Pre-repair status restoration and auditable exchange
Sprint 7 - Employee Lifecycle	Pre-hire onboarding and internal offboarding	Implemented; invitation governance refinement planned	Evidence-backed checklists and closure gates
Sprint 8 - Production hardening	Signing, MSI, update channels, performance, expanded E2E QA	Planned / active backlog	Required before wide endpoint deployment

8.2 Native Agent Delivery Priorities

PRIORITY	COMPLETED	OUTSTANDING
P0 Secure enrollment	Single-use code, unique hashed credentials, binding, audit, rotation/revocation	Retire temporary global-token compatibility after migration
P1 Native service	Heartbeat, policy, inventory, activity, screenshot, USB/DLP	Second-device restart and DLP acceptance
P2 Installation	Install, repair, diagnostics, uninstall, ZIP package	Trusted code signing and signed MSI
P3 Reliability	Recovery, logs, intervals, update verification framework	Production manifest, pilot/stable channels, automatic rollback dashboard
P4 Migration	First pilot deployment and credential validation	Controlled fleet migration and global-token removal

Chapter 9 - Conclusions and Recommended Roadmap

9.1 Conclusions

AstreaBlue demonstrates that a centralized Service Desk can be extended into an enterprise control platform without forcing every business system into one codebase. Human users, external systems, and endpoint devices use different authentication models but converge on a common audited operational record. Asset assignment, consent, endpoint policy, lifecycle, and ticketing are connected through controlled backend workflows rather than manual assumptions.

The strongest project characteristic is its preservation of source-of-truth and access boundaries. External tickets use the existing Service Desk engine. Endpoint identities survive repair and reassignment. Onboarding evidence is reconciled from real account, consent, asset, heartbeat, inventory, and policy records. Replacement updates custody while retaining history. These design choices reduce duplicate data and make management reporting defensible.

9.2 Priority Roadmap

PRIORITY	NEXT ACTION	EXIT CRITERION
1	Complete end-to-end role and branch regression matrix	SuperAdmin, Admin, Technician, Employee, and HR pass positive and negative access tests
2	Finalize onboarding invitation governance	Employee-only HR initiation is backend-enforced or Admin-only policy is documented and tested
3	Complete physical endpoint acceptance	Second laptop passes install, restart, repair, screenshot, USB/DLP, offline queue, and uninstall tests
4	Sign and package the native agent	Trusted certificate, signed EXEs/scripts, signed MSI, verified clean install
5	Retire legacy monitoring token	All active devices use unique credential_last_seen_at evidence; fallback removed
6	Finish modular backend extraction	Route ownership centralized; server.js reduced without duplicate behavior
7	Performance and scale verification	Paginated large lists, indexed queries, measured Railway response times, 120-screenshot gallery test
8	Disaster recovery and operational runbooks	Backup/restore drill, key-rotation procedure, SMTP/R2 outage playbooks

9.3 Final Recommendation

The system is suitable for controlled demonstration and continued pilot use. Broad production rollout of the endpoint agent should wait for trusted code signing, signed installer delivery, multi-device restart acceptance, and retirement of the shared legacy token. The web platform should continue receiving regression-focused improvements, especially around role boundaries, lifecycle invitation governance, pagination, and operational observability.

References

- [1] National Privacy Commission, Republic Act 10173 - Data Privacy Act of 2012.
<https://privacy.gov.ph/data-privacy-act/> (accessed 22 July 2026).
- [2] OWASP Foundation, OWASP API Security Top 10 - 2023, including Broken Object Level Authorization and Broken Function Level Authorization. <https://owasp.org/API-Security/> (accessed 22 July 2026).
- [3] National Institute of Standards and Technology, Security and Privacy Controls for Information Systems and Organizations, NIST SP 800-53 Revision 5, Audit and Accountability family.
<https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final> (accessed 22 July 2026).
- [4] Microsoft, How to Use Data Protection (.NET) and ProtectedData/DPAPI documentation.
<https://learn.microsoft.com/en-us/dotnet/standard/security/how-to-use-data-protection> (accessed 22 July 2026).
- [5] AstreaBlue project repository, backend and frontend source, database migrations, automated tests, and internal technical documentation, repository state dated 22 July 2026.

Appendix A - Demonstration Script

The following sequence is recommended for a thesis defense because it moves from executive visibility to operational proof and finally to governance.

STEP	ROLE	DEMONSTRATION	EXPECTED EVIDENCE
1	SuperAdmin	Open Executive Dashboard and explain common source of truth	KPIs from tickets, assets, endpoints, compliance
2	Employee	Create a branch-scoped ticket with attachment	Canonical ticket number, Open Queue, SLA deadlines
3	Technician	Accept eligible ticket, start work, comment, resolve	Assignment, first response, work tracker, resolution history
4	SuperAdmin	Show external Inventory ticket	INVENTORY-date-sequence number and origin metadata
5	Admin	Show hardware assignment and software licenses	Asset custody, status, history, branch scope
6	Admin	Open CMDB dependency map and impact analysis	Upstream/downstream relationships, impact score
7	Employee/Admin	Run repair or replacement path	Asset state changes and request history
8	SuperAdmin	Show endpoint health, screenshot, USB/DLP	Heartbeat, inventory, consent policy, encrypted media metadata
9	HR/Admin	Open onboarding case and explain automatic evidence	Checklist count matches account/consent/asset/device facts
10	HR/Admin	Open offboarding case	Internal deactivation, asset return, license release, closure gate
11	SuperAdmin	Export a filtered report	Logo, scope, datestamp, table, protected/non-editable output

Appendix B - External Ticket API Example

Production request endpoint

```
POST https://backend-production-fc059.up.railway.app/api/v1/external/tickets
```

Required headers

```
Content-Type: application/json x-api-key: <unique server-side system API key>
```

Example JSON payload

```
{
  "external_employee_id": "INV-EMP-001",
  "requester_name": "Demo Employee",
  "requester_email": "demo.employee@example.com",
  "origin_system": "Inventory System",
  "origin_module": "Stock Management",
  "external_reference": "INV-DEMO-001",
  "title": "Unable to update stock quantity",
  "description": "The stock adjustment page does not save.",
  "priority": "P2-High"
}
```

The credential must be stored only in the external system backend. The browser-facing Help form sends data to that backend, which then calls AstreaBlue. The `external_reference` must be stable so safe retries do not create duplicates.

Appendix C - Operational Acceptance Checklist

AREA	MANDATORY ACCEPTANCE
Authentication	Valid and invalid login, expired JWT, deactivated account
RBAC	Positive and negative cases for every role, branch, ownership, and sensitive category
Ticket lifecycle	Create, assign, start, comment, attach, resolve, close, cancel, and SLA history
External gateway	Valid key, revoked key, invalid payload, replay, conflict, cross-system retrieval
Asset	Create, assign, history, discovery match, depreciation, license expiry/renewal
Replacement	Repair and replacement paths, asset state, endpoint assignment, rollback on invalid state
Consent	Draft, submit, approve, change request, withdraw, effective policy regeneration
Endpoint	Install, restart, heartbeat, inventory, activity, screenshot, USB/DLP, offline retry, repair, uninstall
Lifecycle	Pre-hire creation, invitation, activation, consent, asset, device evidence, completion, offboarding
Reporting	Branch filters, date filters, Excel/TXT/PDF, logo, datestamp, non-editable official output
Resilience	SMTP failure, R2 failure, WebSocket reconnect, database migration/readiness, backup restore

Appendix D - Acronyms and Terms

TERM	MEANING
API	Application Programming Interface
CAB	Change Advisory Board; retained only in legacy change documentation, not the active replacement workflow
CI	Configuration Item
CMDB	Configuration Management Database
DLP	Data Loss Prevention
DPAPI	Windows Data Protection API
HR	Human Resources role with restricted lifecycle oversight
ITSM	Information Technology Service Management
JWT	JSON Web Token
RBAC	Role-Based Access Control
R2	Cloudflare object storage used for private encrypted screenshots
SLA	Service Level Agreement
UUID	Universally Unique Identifier used for durable endpoint identity